Übungsblatt 6: Multicore-Echtzeit und Shared Memory

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie begründen Kernrollen, Speicherlayout, Besitz und Sichtbarkeit. Grundlage sind Vorlesungen 1–6. AMP, Asymmetric Multiprocessing, bedeutet hier getrennte Firmwarekontexte; SMP, Symmetric Multiprocessing, eine gemeinsame Betriebssystemverwaltung mehrerer Kerne. MPU bedeutet Memory Protection Unit.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	20	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **85 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 75 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten. Im Papierkern sind 15 Minuten für Aufgabe 1, 15 Minuten für Aufgabe 2 und zusammen 20 Minuten für Aufgaben 3–4 vorgesehen.

Teil A: Papieraufgaben

Aufgabe 1: Softwaretrennung und Zeitbudget · 8 Punkte

1. Die M4-Firmware verwendet einen lokalen FreeRTOS-Kernel, die M7-Firmware eine eigene Hauptschleife. Ist dies das heutige AMP- oder ein SMP-Modell? Darf der M7 eine im gemeinsamen Speicher abgelegte M4-Queue-ID direkt verwenden? Schützt ein M4-Mutex automatisch vor M7-Zugriffen? Begründen Sie jede Aussage. (4 P)
2. Lehrmodell: Eine Reaktion benötigt 4 ms eigene M4-Arbeit, zusätzlich 3 ms Warten auf ein benötigtes M7-Ergebnis und 2 ms Businterferenz. Die Anteile sind ausdrücklich nicht überlappend. Relative Deadline: 8 ms. Berechnen Sie Antwortzeit und Fristabweichung. Warum reicht das Wissen um die eigene M4-Arbeit nicht aus? (2 P)
3. Beide Kerne konfigurieren verschiedene Pins desselben GPIO-Ports durch Lesen, Ändern und Zurückschreiben des gemeinsamen Konfigurationsregisters. Warum genügt die getrennte Pinzuordnung in CubeMX nicht? Nennen Sie eine geeignete Zuständigkeitsregel. (2 P)

Musterlösung Aufgabe 1

Die getrennten Firmware- und Kernelkontexte sind AMP. Eine Queue-ID ist ein lokaler Handle der M4-Verwaltung; Wartelisten, Speicherverwaltung und Kernelaufrufe werden nicht durch den Ablageort zu M7-Funktionen. Der M7 benötigt einen ausdrücklichen Kanalvertrag. Ebenso wirkt der M4-Mutex nur für Teilnehmer dieser Schutzregel und schließt den Bare-Metal-M7 nicht automatisch aus.

$R = 4 + 3 + 2 = 9\,\text{ms}$. Die Deadline wird um $9 - 8 = 1\,\text{ms}$ verletzt. Eigene Arbeit ist nur ein Antwortanteil, nicht die gesamte Antwortzeit. Wäre die Busverzögerung bereits in einer anderen Größe enthalten, dürfte sie nicht nochmals addiert werden. Diese Summe gilt nur für das vorgegebene Modell.

Pin Context steuert die Codegenerierung, keine Hardware-Zugriffssperre. Verschiedene Pins können Felder desselben Registers teilen. Zwei Read-Modify-Write-Folgen können eine Änderung gegenseitig überschreiben. Geeignet ist ein Besitzer für das gemeinsame Konfigurationsregister beziehungsweise den Peripherieblock. Gemeinsame Takte initialisiert im Basis-Bootvertrag der M7, bevor der M4 normal weiterläuft.

Aufgabe 2: Layout, Start und Phasenbesitz · 8 Punkte

Gemeinsamer C-Typ aus der Vorlesung:

```
typedef struct {
    uint32_t magic;
    uint32_t state;
    uint32_t payload;
} ipc_block_t;
```

Jedes Wort hat vier Byte, natürliche Ausrichtung vier Byte. Das Objekt `ipc` wird auf 32 Byte ausgerichtet und seine Sektion auf 32 Byte gepolstert. SRAM4 beginnt bei `0x38000000` und besitzt 64 KiB. Die Kennung wird vor dem M4-Start gesetzt.

1. Geben Sie Feldoffsets, Typgröße, Sektionsreserve und MPU-Regionsgröße an. Warum sind die drei Größen verschieden? (2 P)

2. Die CM7-Map nennt `ipc` bei `0x38000000`, die CM4-Map bei `0x38000020`. Beide verwenden denselben Header. Ist das ein funktionsfähiger gemeinsamer Block? Was muss angepasst und geprüft werden? (2 P)
3. Beginnen Sie mit Payload null und `REQUEST_M4`. Bei jeder Antwort erhöht der M4 um eins. Vor der nächsten Anfrage erhöht der M7 ebenfalls um eins. Notieren Sie drei Antwortwerte, die beiden weiteren Anfragewerte und den endlichen Abschlusszustand. Nennen Sie das Schreibrecht je Phase und die Barrierepositionen. (2 P)
4. Eine Sektion ist `NOLOAD`, das M4-Startup nullt aber trotzdem einen überlappenden Bereich nach dem Aufwachen. Warum ist das falsch? Geben Sie die gültige Init-/Freigabereihenfolge und den Umgang mit einseitigem Core-Reset an. (2 P)

Musterlösung Aufgabe 2

Offsets: magic null, state vier, payload acht. Typgröße zwölf Byte, gepolsterte Sektion 32 Byte, MPU-Fenster 64 KiB beziehungsweise 65536 Byte. Die Typgröße beschreibt die Felder, die Sektionsreserve isoliert und richtet das Objekt aus, die MPU beschreibt eine größere Region. Objektalignment ändert nicht automatisch die natürliche Typgröße.

Die Maps bezeichnen verschiedene Bytes. Gleicher Name und Header reichen nicht. Beide tatsächlichen Linkerdateien müssen das einzige Objekt an denselben vereinbarten Beginn legen. Prüfen Sie die ausgewählte Datei im CMake-Linkaufruf, Sektion, Assertions, andere Allokationen und beide resultierenden Maps. Private Daten, Heap und Stack liegen außerhalb der reservierten SRAM4-Bank.

Aktion	Payload danach	Zustand danach	Privater M7-Antwortwert
M7 initialisiert	0	REQUEST_M4	–
M4 antwortet	1	RESPONSE_M7	–
M7 liest und veröffentlicht	2	REQUEST_M4	1
M4 antwortet	3	RESPONSE_M7	–
M7 liest und veröffentlicht	4	REQUEST_M4	3
M4 antwortet	5	RESPONSE_M7	–
M7 beendet	5	TURN_M7	5

Bei `REQUEST_M4` darf der M4 die Payload verändern, bei `RESPONSE_M7` der M7. Der vorige Besitzer schreibt nach Übergabe nicht weiter. Veröffentlichung: Payload schreiben, DMB, Zustand schreiben. Konsum: erwarteten Zustand prüfen, DMB, Payload lesen. DMB ist Data Memory Barrier, Datenspeicherbarriere. Genau ein Teilnehmer je Kern wird vorausgesetzt.

`NOLOAD` stellt keine normalen Initialdaten für diese Sektion bereit, verbietet aber nicht beliebige Startup- oder Debugger-Schreibzugriffe. Beide Copy-/Zero-Tabellen müssen `.shared` aussparen. Der M7 richtet den Block einmal ein, während der M4 im vereinbarten Stop-Bootzustand wartet, und gibt ihn erst danach frei. Einseitiger Reset während einer Runde gehört nicht zum Basisvertrag. Eine alte gültige Kennung ist keine neue Sitzung; Wiederanlauf erfolgt hier nach Diagnose mit gemeinsamem Neustart.

Aufgabe 3: Sichtbarkeit und ganze Cachezeilen · 8 Punkte

Die folgenden Werte sind synthetische Zustände, keine Boardmessungen. Der M7-Daten-Cache besitzt 32-Byte-Zeilen, der M4 kann ihn nicht direkt lesen.

1. M7 schreibt eins nur in eine Write-Back-Kopie, SRAM enthält weiterhin null. Welchen Wert kann M4 lesen? Umgekehrt schreibt M4 eins in SRAM, während M7 eine alte saubere gültige Kopie mit null hat. Was kann M7 lesen? Behebt `volatile` plus DMB beide Fälle? Begründen Sie. (4 P)
2. Ein Puffer beginnt bei `0x38000014` und hat 16 Byte Länge. Bestimmen Sie abgerundeten Wartungsbeginn, aufgerundetes exklusives Ende, Bytezahl und Zahl der berührten Zeilen. (2 P)
3. In einer Zeile liegen die Wörter A und B. Start beider null. M7 ändert in seiner Kopie A auf eins; die Zeile ist dirty, B dort noch null. M4 schreibt neun in B des SRAM. Was kann ein anschließendes Clean der alten M7-Zeile mit B tun? Welche Granularität muss die Besitzregel bei Cachewartung berücksichtigen? (2 P)

Musterlösung Aufgabe 3

M4 kann den alten SRAM-Wert null lesen. M7 kann seine alte Cachekopie mit null lesen, obwohl SRAM bereits eins enthält. `volatile` veranlasst den vorgesehenen Compilerzugriff, ein Load kann aber weiterhin aus dem Cache kommen. DMB ordnet relevante Zugriffe und schreibt Dirty-Zeilen nicht automatisch zurück. Clean schreibt geänderte Zeilen zurück, Invalidate verwirft Kopien. Invalidate eigener Dirty-Daten könnte unveröffentlichte Änderungen verlieren.

Die Startadresse liegt zwanzig Byte nach `0x38000000`. Sechzehn Byte Nutzlänge enden exklusiv bei `0x38000024`, also bereits in der zweiten Zeile. Wartungsbeginn: `0x38000000`, exklusives Ende: `0x38000040`, Spanne 64 Byte, zwei Zeilen. Nachbarobjekte dieser Spanne sind mitzudenken.

Die M7-Kopie enthält für A/B eins/null, SRAM nach dem M4-Schreiben null/neun. Das Clean der Dirty-Zeile kann SRAM auf eins/null überschreiben und damit die neuere neun verlieren. Zwei getrennte Wörter sind keine getrennten Cachezeilen. Wartung braucht exklusive Zuständigkeit für die vollständigen betroffenen Zeilen und einen sicheren Veröffentlichungs-/Konsumvertrag, auch für die Kontrollinformationen.

Aufgabe 4: MPU, Regionspriorität und Fehlerbudget · 8 Punkte

1. Region vier beschreibt SRAM4 ab `0x38000000` mit 64 KiB als Normal, TEX eins, C null, B null, S eins, Lesen/Schreiben und Execute Never. Region sechs überlappt die erste Hälfte als cachebares Normal-Write-Back mit S null. Beide Regionen und alle betreffenden Subregionen sind aktiviert. Welche Region gilt für `ipc` am Anfang? Erzeugt S eins einen M7/M4-Cacheabgleich? Schützt die M7-MPU damit automatisch vor M4- oder DMA-Zugriffen? (3 P)
2. Bestimmen Sie die letzte eingeschlossene Adresse der 64-KiB-Region und den Armv7-M-SIZE-Code mit Größe $= 2^{\mathrm{SIZE}+1}$ Byte. Welche Reihenfolge gilt zwischen MPU-Konfiguration, D-Cache-Aktivierung und Kommunikation? (2 P)
3. M4 wartet im Modul bis zu 2000 Kernel-Ticks, M7 prüft 2000 HAL-Millisekunden. Bei bestätigten 1000 Kernel-Ticks/s: Welche nominellen Budgets sind das? Warum beweisen sie keine zwei Sekunden vollständige IPC-Deadline? Darf der Fehlerpfad den Shared-Block einfach nullen? (3 P)

Musterlösung Aufgabe 4

Die passende höher nummerierte Region sechs gewinnt. Die beabsichtigte nicht cachebare Region vier allein ist deshalb kein Nachweis der effektiven Attribute. S bedeutet shareable und aktiviert auf diesem System keinen automatischen M7/M4-Cacheabgleich. Die M7-MPU ist kernlokal und kontrolliert nicht automatisch den anderen Kern oder DMA, Direct Memory Access beziehungsweise direkten Hardware-Speicherzugriff.

$64 \cdot 1024 = 65536 = 2^{16}$ Byte. Die letzte eingeschlossene Adresse ist `0x3800FFFF`, SIZE ist fünfzehn. Am gemeinsamen Neustart zuerst die Region vor D-Cache-Aktivierung konfigurieren. Im kontrollierten Versuch wird der Block bei D-Cache aus initialisiert, dann D-Cache eingeschaltet und der M4 freigegeben. Die laufende Nutzung wird nicht durch spontane Attributumschaltung verändert.

Nominell 2000/1000 = 2 Sekunden M4-Tickbudget und 2000 ms gleich zwei Sekunden M7-Budget. Die tatsächliche Prüfung setzt laufende Zeitbasen und erneute Ausführung des Wartecodes voraus. M4-Start, Scheduling, andere Unterbrechungen und die definierte Gesamtreaktion sind damit nicht automatisch vollständig begrenzt; Tickzeit ist kein unabhängiger Watchdog. Bei Timeout wird lokal diagnostiziert und angehalten, nicht ohne Besitz `memset` ausgeführt. Die Gegenstelle könnte noch arbeiten. Der erneute Basisversuch benutzt gemeinsamen Reset.

Transfer: Vier Aussagen im Entwurfsreview · 20 Minuten

Prüfen Sie zehn Minuten einzeln und besprechen Sie zehn Minuten. Keine zusätzlichen Punkte. Auf dem H755 werden ein nicht cachebarer Shared-Bereich und ein M4-Kernel verwendet. Es gibt keine automatische Schutzwirkung über beide Kerne.

1. „Nicht cachebar macht `counter++` zwischen M7 und M4 unteilbar.“ Widerlegen oder bestätigen Sie die Aussage mit einem Maschinenablauf ab Zählerstand 40.
2. „Ein lokaler M4-Mutex um den Zähler genügt auch für den Bare-Metal-M7.“ Welche Teilnahme fehlt?
3. „Nach einem Timeout darf M7 den ganzen Block neu initialisieren.“ Welche Besitz- und Wiederanlaufbedingung fehlt?
4. Eine obere Antwortgrenze von 7 ms enthält ausdrücklich schon 2 ms Busverzögerung. Ein Review addiert nochmals dieselben 2 ms. Ist 9 ms damit eine korrekt zerlegte Grenze? Was wäre zu tun, wenn die Buslastannahme geändert wird?

Musterlösung Transfer 6

Beide Kerne können 40 lesen, privat 41 berechnen und 41 schreiben; erwartet wäre 42. Nicht cachebar betrifft Sichtbarkeit, nicht die Unteilbarkeit einer mehrteiligen Operation. Der M7 nimmt am lokalen Kernelmutex nicht teil. Ein gemeinsamer Besitzvertrag, beispielsweise passend eingesetzter HSEM plus lokale Serialisierung, ist nötig.

Timeout beweist nicht, dass der andere Schreiber aufgehört hat. Im Basisvertrag erfolgt keine einseitige Neuinitialisierung; beide Kerne werden kontrolliert gemeinsam neu gestartet und der Block vor Freigabe einmal initialisiert.

Die 9 ms können als unnötig größere Zahl weiterhin oberhalb der angegebenen 7-ms-Grenze liegen, sind aber keine korrekte Zerlegung der Kosten: Derselbe Anteil wird doppelt gezählt. Mit unveränderten Annahmen bleibt die angegebene Grenze 7 ms. Bei anderer Buslast muss der betroffene Anteil beziehungsweise die gesamte Analyse unter neuen Annahmen neu begründet werden; nicht eine alte Mess- und Modellgröße blind aufsummieren.

Teil B: Praxisfall · 15 Minuten

Praxisfall 6: Beide Sichten nebeneinander

Papieralternative, explizites Cachemodell: Der Phasenkanal hat genau einen Teilnehmer je Kern. `REQUEST_M4` übergibt an M4, `RESPONSE_M7` an M7. M7 besitzt eine gültige Write-Back-

Kopie; es findet keine Cachewartung statt. Nach der ersten Antwort und einer lokal geschriebenen Folgeanfrage liegt der folgende Zustand vor:

Sicht	state	payload
M7-Cache, Dirty-Kopie	REQUEST_M4	2
SRAM, daraus liest M4	RESPONSE_M7	1

1. Worauf warten beide Teilnehmer? Kann allein DMB die Differenz beheben? Belegt die SRAM-Debugansicht den Wert eines laufenden M7-Loads?
2. Vergleichen Sie mit einem korrekt nicht cachebaren, geordneten Kanal: Startpayload null, M4 erhöht pro Antwort um eins, M7 vor jeder Folgeanfrage ebenfalls. Welche drei Antworten und welcher finale Payload entstehen?
3. Was ist bei Timeout erlaubt? Weshalb wird die Integration eines neuen Linker-/MPU-Layouts nicht für diesen Erklärversuch verlangt?

Bei vorbereitetem Board: Die Lehrperson zeigt den nicht cachebaren Basislauf mit Antworten 1, 3, 5. Ein cachebarer Fehlerbuild ist nur mit eigener vorab geprüfter Projektvariante sinnvoll; der Fehler muss nicht in jedem Lauf sichtbar werden. Der Papierfall bleibt die verlässliche Erklärung.

Musterlösung Praxisfall 6

M7 wartet auf eine Antwort zur neuen Anfrage; M4 sieht noch die alte Antwortphase und wartet auf die nächste Anfrage. DMB ordnet Zugriffe, schreibt die Dirty-Kopie aber nicht automatisch zurück. Ein Debuggerzugriff auf SRAM zeigt diese Sicht, nicht notwendigerweise die verwendete M7-Cachekopie.

Im gültigen geordneten Kanal sind die Payloadschritte 0, 1, 2, 3, 4, 5; Antworten 1, 3, 5. Nach der dritten Antwort wird kein neuer Request veröffentlicht und Payload bleibt fünf. Die genaue Endphase ist `TURN_M7` wie im Modulvertrag. Timeout erlaubt Diagnose, aber kein Nullen bei möglicherweise arbeitendem Partner. Für einen neuen Basislauf gilt gemeinsamer Neustart. Das Sichtbarkeitskonzept lässt sich ohne laufenden Generator, Linker oder Debugger aus beiden vorgegebenen Sichten erklären.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 6; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; ST RM0399, UM2408, AN5617, AN4838 und AN5361; eingebundene STM32H7-HAL- und CMSIS-M7-Header sowie die Dokumentation der installierten STM32 VS Code Extension. Das öffentlich eingesehene H745-Boottemplate illustriert das Muster; die H755-Projektbasis und ihre Versionen bestimmen die tatsächliche Integration.